



中国地质大学

CHINA UNIVERSITY OF GEOSCIENCES

北京 · BEIJING

导师制实习报告 (竞赛集训)

姓名: 黄恺瑞
学号: 1006202211
专业: 计算机科学与技术
课程名称: ACM 竞赛集训
集训内容: ACM 竞赛相关算法的学习和指导
指导老师: 张玉清
提交时间: 2021 年 7 月

目录

一、个人总结.....	1
二、算法总结.....	2
1 搜索.....	2
1.1 盲目搜索.....	2
1.2 记忆化搜索.....	3
1.3 二分查找、二分答案.....	4
1.4 三分查找.....	5
1.5 搜索的优化.....	5
2 动态规划.....	6
2.1 背包问题.....	6
2.2 最长公共子序列 (LCS)	7
2.3 最长不下降/上升子序列(LIS)	9
2.4 最长公共子串.....	10
2.5 最长回文子串.....	10
2.6 Manacher 算法	11
3 数据结构.....	12
3.1 并查集.....	12
3.2 树状数组.....	13
3.3 线段树.....	13
3.4 ST 表.....	15
3.5 分块.....	16
3.6 莫队算法.....	18
4 图论.....	20
4.1 图论综述.....	20
4.2 拓扑排序.....	21
4.3 最小生成树.....	22
4.4 二分图匹配.....	24
4.5 最短路问题.....	26
4.6 Tarjan 算法	28
5 数论.....	30
5.1 最大公因数(GCD)	30
5.2 最小公倍数(LCM)	30
5.3 素数.....	30
5.4 欧拉函数.....	33
5.5 组合数.....	34
5.6 快速幂.....	35
5.7 扩展欧几里得.....	35
5.8 除法取模与逆元.....	36
5.9 威尔逊定理.....	37
6 STL 库	37
6.1 unique 函数	37
6.2 map 的遍历.....	38
6.3 lower_bound&upper_bound	38
6.4 其他 STL	39

一、个人总结

去年刚进入学校就努力进入 ACM 队，经过吴成琦队长简单的面试之后，留下了 CUGBACM200 题的任务，靠这个 200 题让自己慢慢捡起了三年前的 OI 基础，最终在十一月初进入 ACM 队。在进队前有一次学校对于程序设计天梯赛的选拔考试，那时候的我很多都忘记了，不出意外的拿到了很差的成绩，这次考试对我的打击很大，努力刷题进入 ACM 队。刚进入 ACM 随 19 级训练，不适应加上自己能力不够，当然很长时间都是在“罚坐”，但是也并没有气馁，努力再学一遍以前的算法等知识，最后在这个学期勉强能够更上队内的节奏。

在校的学期内训练，是周一周二晚上/周三周四晚上/周五周六晚上/周日下午进行训练；在学期外的训练，是从上午三小时，下午三小时，晚上三小时，全天九个小时的训练。队内 cugbadmin 给的训练是基于 vjudge 上的 contest 比赛，当然队内几个队员也会组织给自己“加训”，在 Codeforces 上的 div2/div3 基本上是必打的，这个时间基本上是在晚上十点半左右，时常一般是两个小时或者两个小时十五分钟。除此之外，我与队伍内其他两位队员参加了 AcWing 的每日一题，还会在不影响训练的情况下参加晚上的一些比赛，像牛客(nowcoder)、和 AcWing 等会有一些周赛月赛等。对于每次比赛我都积极总结，对于好的题目单独拿出来放入分栏，会总结与反思一些注意事项、细节、考试时的思想等。我的博客网址是 https://blog.csdn.net/Carry_hkr?spm=1011.2124.3001.5343，会记录自己的每日一题，每次的比赛总结，经典题目，板子，技巧等。接下来是我对比赛做题的总结：

保证自己的心态：我将心态方面分为两种，一种是自己做不出题目的时候的心态，不能放弃自己，有的时候做很长时间却仍然无法 AC，这时候也许是最崩溃的，也不想放弃这个题，又有点不想考了，这个时候我们就要调节自己的心态，也许排名降低了但也只是一时的，也许最后不在前列，但也可以上升很大阶段。还有一种心态就是自己是大一的心态，不能认为自己时间还长，或者本应该就比学长们成绩低的心态，一直在跟 19 级训练就不应该认为自己是 20 级学生，给自己一定的压力，不然早就在比赛前就放弃自己了。

团队比赛时候多交流：经过一次西安站的邀请赛还有一些网络的团队比赛之后，我和我的队友有了一定的分工，我认为这个分工一定要明确，可以重合，但不能有缺漏。例如我负责数据结构，A 负责图论，B 负责动态规划等，但是 A 和 B 都会数据结构，我和 A 都会动态规划，这是不错的场面，范围要囊括尽量大的方面。有一次线上一场其他学校校赛，最后一道题目我看了看觉得应该是前段时间刚做的分块类型的题目，然后队内的一个人看了之后说这应该用莫队算法可以过，我恍然大悟，然后没过多久就 AC 了这个题，这说明我们团队的分工要明确，考试的时候多交流，多讨论。在邀请赛中，一名队友说自己代码能力不太好，让我一起看着他写这个题，我们两一起纠正，边写边讨论纠错，但是代码能力一定要提升。

平时做好准备：平时训练要做好总结，自己当时为什么要错，是怎么往错误方向思考的，这个题目是有什么样的细节等等。对于板子用法，我认为应当少用，少用对于提升自己代码的能力有很大的提升。举个例子，对与一个简单的二分答案，有很多人不知道边界怎么设置，具体的是如何赋值，有的时候是 $r = mid - 1$ 有的时候是 $l = mid + 1$ ，有的时候是 $mid = (l + r) >> 1$ ，有的时候是 $mid = (l + r + 1) >> 1$ ，大部分人都是自己一个一个去试，但是我们少用板子，自己去思考一下是求左边界还是右边界，答案很容易出来。到时候用板子用错了，一个劲在那纠错很浪费时间，不如自己平时做好准备，做多了不用几眼就能看出来问题出在哪。

这个学期的成绩还算不错，校赛中拿到了金牌第三的成绩；程序设计天梯赛进入了一队，团体拿到了全国一等奖，北京市特等奖；蓝桥杯拿到了省赛一等奖，国家二等奖成绩；ICPC 邀请赛西安站打了铁...希望自己不要气馁，再接再厉，争取年年进步，最后达到自己满意的成绩。在今后的 ACM 生涯中努力学习，虚心学习，提升自己的算法与数据结构的能力，同样也要提高自己的英语能力，在读题上无大障碍，好好反思，好好总结。

二、算法总结

1 搜索

1.1 盲目搜索

1.1.1 纯盲目搜索

盲目搜索适用于深度不大，打暴力、对拍的时候，能搜出所有情况。比如一个长度为 20 的数组，题目简化为选其中 m 个进行处理，那么这个盲目搜索对于 20 位进行选还是不选的操作搜索下去，每一种情况都能得到。但是时间复杂度是指数级别的，对于大数据来说，不用记忆化搜索是很难搜完的。

1.1.2 深度优先搜索

深度优先搜索(Depth First Search)，顾名思义是按照深度优先，不同于广度优先搜索，对于一个节点 V_0 ，它将按照深度优先往下一直走到底，直到不能走则返回上一个，然后从另外一条路走到底。深度优先搜索适用于图的遍历、拓扑排序等题目，在 OI 中利用深搜可以搜出很多分数，但是在 ACM 赛制中纯搜索答案很难 AC。

```
1. void dfs(int x)
2. {
3.     if (num==n)
4.     {
5.         ...
6.     }
7.     for (int i=head[x];i=edge[i].next)
8.     {
9.         int y=edge[i].to;
10.        if (exist[y]) continue;
11.        ...
12.        dfs(y);
13.        ...
14.    }
15. }
```

想要优化的话，我们可以进行剪枝操作，对于搜到的当前的答案，如果大于已求的答案，就没有必要再走下去了，毕竟再往下走也不是最优解，我们可以在每一次的 dfs 前加上剪枝。

```
1. void dfs(int x)
2. {
3.     if (tmp>=ans) return;
4.     ...
5. }
```

1.1.3 广度优先搜索

广度优先搜索(Breadth First Search)，又称宽度优先搜索，不同于深搜的是它会先搜完这一层再搜索下一层。换句话说，它并不考虑结果的可能位置，彻底地搜索整张图，直到找到结

果为止。BFS 适用于 Dijkstra 求最短路，还有最小生成树 Prim 算法等，适用于题目问一些“步数最少”，“操作最少”等问题。广搜有时可能会搜重，一个点重复判断，这时候就要看所用的算法了，比如 Dijkstra 是只进入一次，但是 SPFA 就不一定。

```
1. queue<int> Q;
2. ...
3. while (!Q.empty())
4. {
5.     ...
6.     exist[u]=false;
7.     for (int i=head[u];i=edge[i].next)
8.     {
9.         ...
10.        if (...)
11.        {
12.            ...
13.            if (!exist[v]) {Q.push(v);exist[v]=true;}
14.            //重新进入队列中
15.        }
16.    }
17. }
```

1.2 记忆化搜索

1.2.1 记忆化搜索实现

记忆化搜索实际上是递归来实现的，但是递归的过程中有许多的结果是被反复计算的，这样会大大降低算法的执行效率。

而记忆化搜索是在递归的过程中，将已经计算出来的结果保存起来，当之后的计算用到的时候直接取出结果，避免重复运算，因此极大的提高了算法的效率。

记忆化搜索与动态规划联系很大，我们可以通过我们搜索传递的参量来写出动态转移方程，或者就用 dfs 改编而来的记忆化搜索来搜的答案。

1.2.2 01 背包加强版

题目描述：一个 01 背包经典问题，加上后面加入的物品不能大于上一个物品。

思路：设置一个类似于 DP 的多维数组，其实就是与 DP 状态转移方程类似，如果当前状态我们已经搜到过，那么就没有必要再往下搜了，这可以节省大量的时间，从而达到 DP 一样的时间复杂度。

```
1. int dfs(int x,int cost,int last)
2. {
3.     if (x>n) return 0;
4.     if (f[x][cost][last]) return f[x][cost][last];
5.     f[x][cost][last]=dfs(x+1,cost,last);
6.     if (v[x]<=v[last] && cost+v[x]<=m)
7.         f[x][cost][last]=Max(f[x][cost][last],dfs(x+1,cost+v[x],x)+val[x]);
```

```

8.     return f[x][cost][last];
9. }

```

1.2.3 路径计数

题目描述：从一个入度为0的点到出度为0的点一共有多少条。

思路：这个题目原本是洛谷上的题，最先做是在小学期结课个人赛上的最后一题，一开始打了个纯暴力的DFS，没有注意到数据范围，最后超时了。注意到如果搜到了某一个点之后，也可能重复有边指向这一个点，那么这个点又会向下深搜，那么这个时间复杂度是难以想象的大，但是如果我们用一个数组将当前状态的答案存下来，下一次搜寻到这个状态的时候我们只需要返回这个答案，那么就能大大减少用时，这样每一条边只会搜索一次。

这是原本的纯暴力dfs：

```

1. inline int dfs(int x)
2. {
3.     if (dfss[x]) return dfss[x];
4.     if (!out[x]) { dfss[x]=1;return 1;}
5.     for (int i=head[x];i;i=edge[i].next)
6.     {
7.         if (!dfss[x])
8.             dfss[x]=dfs(edge[i].to);
9.         ans=(ans+dfss[x])%M;
10.    }
11. }

```

在dfs基础上修改成记忆化搜索之后的dfs

```

1. inline int dfs(int x)
2. {
3.     if (f[x]) return f[x];
4.     int t=0;
5.     for (int i=head[x];i;i=edge[i].next)
6.     {
7.         int y=edge[i].to;
8.         t=(t+dfs(y))%M;
9.     }
10.    if (t==0) t=1;
11.    f[x]=t;
12.    return f[x];
13. }

```

1.3 二分查找、二分答案

1.3.1 二分查找

二分查找(Binary Search)，是在有一定单调性下的搜寻答案的快速方法，时间复杂度往往是 $O(\log n)$ 的。如同小时候玩的猜数字游戏，在心中想一个1..100的一个数字，另外一

个小伙伴猜一个数,然后告诉他是大还是小了,这样继续下去,最少是 $\log$ 级别的可以猜出。

在 STL 库中有现成的 `binary_search` 函数,需要 `algorithm` 库,具体见后面的 STL 部分。

1.3.2 二分答案

二分答案往往对于降低时间复杂度有奇效,对于一个具有单调性的答案,我们可以用二分查找的方法去判断 `mid` 是否可行,在具体题目具体判断,怎么样对 `l` 和 `r` 进行取值,最终我们得到的答案时间复杂度也是 $O(\log n)$ 的。

二分答案一般适用于问题类似于“求最大值最小化”、“求最小值最大化”。经典题目有跳石子等。

```
1.  bool judge(int x)
2.  {
3.      ...
4.      if (...) return true;
5.      else return false;
6.  }
7.  while (l<r)
8.  {
9.      int mid=(l+r+1)>>2;
10.     if (judge(mid)) l=mid;
11.     else r=mid;
12. }
```

对于 `l` 和 `r` 的操作是否什么时候直接等于 `mid`, 是否需要 `mid - 1` 或者 `mid + 1`, 这都要视题目具体情况而定的,看你是否查找左侧边界还是右侧边界,是需要 `[left, mid - 1]` 还是 `[mid + 1, right]`。

注意一些细节,建议 `mid` 的取值不是直接 `l + r` 再除以 2, 这样容易 `l + r` 越界,那么我们可以 `mid = l + (mid - l)/2`, 这样不会越界; 建议不要出现 `if` 后直接 `else`, 建议出现 `else if`, 把等于的情况也讨论进来, 这样不容易出错。

1.4 三分查找

三分查找做法类似于二分查找, 要求区间内只有一个极值, 时间复杂度 $O(\log_3 n)$, 设置 $mid = \frac{(left+right)}{2}$, $midmid = \frac{(mid+right)}{2}$, 然后如果 $f(mid) < f(midmid)$ 那么 `left = mid`; 如果 $f(mid) > f(midmid)$ 那么 `right = midmid`, 终止条件是 $|left-right| < eps$ 。

1.5 搜索的优化

1.5.1 剪枝

在前面的 `dfs` 内容稍微提及了一下, 如果当前状态并且此时的值已经大于等于我们现在的 `ans`, 那么我们就将此刻的 `dfs` 这个“树枝”直接“剪断”, `return` 掉。

```
1.  void dfs(int x)
2.  {
3.      if (tmp>=ans) return;
4.      ...
5.  }
```

1.5.2 记忆化搜索

上述 1.2 已经讲述的很详细了，是一种不错的方法，大大提高效率。

1.5.3 判重

如果遇到重复经过的，没有再重新走的必要，那么我们就不要再往下搜索了，这一点跟记忆化搜索很相似。

1.5.4 clock 函数（下下策）

这个需要看运气碰巧说不定可以。题目假如限制1s，那么我们搜索的答案对于全局来说会得到一个，但是也许并不是最优解，也许就是最终的答案。那么我们假如已经到达了985ms了，我们也许此时的答案早已搜到，在之后的答案也许一直在做无用功，那么我们可以直接输出此时的答案然后结束整个程序。这种方法是下下策，不到万不得已不要使用，毕竟这不是 ACM 赛制，要看很大的运气，对于得到部分分还是能够拿到一定的分数。但也不是完全不可以，在某一次天梯赛训练题中一道图论题就是用卡时间的方法 A 掉的。

2 动态规划

2.1 背包问题

2.1.1 01 背包问题

题目：有 n 件物品和一个容量为 V 的背包。第 i 件物品的费用是 $w[i]$ ，价值是 $v[i]$ ，求将哪些物品装入背包可使价值总和最大。

思路：这种题目就是01背包问题，要么放，要么不放，且如果放也只有一个，我们定义 $f[i][j]$ 表示前 i 件物品恰放入一个容量为 j 的背包可以获得的最大价值。则其状态转移方程是： $f[i][j] = \max(f[i-1][j], f[i-1][j-w[i]] + v[i])$

这个 f 数组是一个二维数组，我们不难发现每一层只需要上一层的数据，那么循环下来大部分上面数据是没有用的，我们只需要最后一行数据来寻找答案，所以我们可以缩短成一维数组，并且从后往前，这样不影响上一层数据。

```
1. for (int i = 1; i <= n; i++)
2.     for (int j = V; j >= 0; j--)
3.         f[j] = max(f[j], f[j - w[i]] + v[i]);
```

对于上面还可以进行优化，我们不难发现第二层循环最后面小的数字是没有用了，我们可以设置下限 $bound$ 变量控制的是最小费用，寻找可以用前缀和优化。

2.1.2 完全背包问题

题目：有 n 件物品和一个容量为 V 的背包。第 i 件物品的费用是 $w[i]$ ，价值是 $v[i]$ ，求将哪些物品装入背包可使价值总和最大。每种物品可以装无限个。

思路：与01背包不同的是，每种物品可以装无限个，那么我们在01背包基础上多加一个 k 个，循环 k 个去更新 f 数组。

状态转移方程： $f[i][j] = \max(f[i-1][j - k * w[i]] + k * v[i] \mid 0 \leq k * w[i] \leq j)$

完全背包可以进行二进制优化：更高效的转化方法是：把第 i 种物品拆成费用为 $w_i * 2^k$ 、价值为 $v_i * 2^k$ 的若干件，其中 k 满足 $w_i * 2^k \leq V$ 。这是二进制的思想，因为不管最优策略选几件第 i 种物品，总可以表示成若干个 2^k 件物品的和。

也可以优化成 $O(VN)$ ：

```
1. for (int i = 1; i <= n; i++)
2.     for (int j = w[i]; j <= V; j++)
3.         f[j] = max(f[j], f[j - w[i]] + v[i]);
```

现在完全背包的特点恰是每种物品可选无限件，所以在考虑“加选一件第 i 种物品”这种策略时，却正需要一个可能已选入第 i 种物品的子结果 $f[i][j - w[i]]$ ，所以就可以并且必须采用 $j = 0 \dots V$ 的顺序循环。

2.1.3 多重背包

题目：有 N 种物品和一个容量为 V 的背包。第 i 种物品最多有 $p[i]$ 件可用，每件费用是 $w[i]$ ，价值是 $v[i]$ 。求解将哪些物品装入背包可使这些物品的费用总和不超过背包容量，且价值总和最大。

思路：这个题目跟前面两种类似，但每一种物品有 $p[i]$ 限制了，那么还是设置一个 k 来枚举数量。状态转移方程是： $f[i][j] = \max(f[i-1][j - k * w[i]] + k * v[i])$ $0 \leq k \leq p[i]$ 。

也可以按照上思路转化成二进制优化。

2.1.4 多组背包

题目：有 N 件物品和一个容量为 V 的背包。第 i 件物品的费用是 $w[i]$ ，价值是 $v[i]$ 。这些物品被划分为若干组，每组中的物品互相冲突，最多选一件。求解将哪些物品装入背包可使这些物品的费用总和不超过背包容量，且价值总和最大。

思路：设置一层循环放外面，这个循环意义是每个组，这样可以保证每组只选择一个。

```
1. for (所有的组 k)
2.     for (int j = V; j >= 0; j--)
3.         for (所有属于组 k 的 i)
4.             f[j] = max{f[j], f[j - w[i]] + v[i]}
```

2.1.5 依赖背包

题目：这种背包问题的物品间存在某种“依赖”的关系。也就是说， i 依赖于 j ，表示若选物品 i ，则必须选物品 j 。为了简化起见，我们先设没有某个物品既依赖于别的物品，又被别的物品所依赖；另外，没有某件物品同时依赖多件物品。（经典例题 *NOIP2016* 金明的预算，博客里面有）

思路：其实依赖背包类似于分组背包，主件和附件是一组的。

2.2 最长公共子序列（LCS）

2.2.1 输出长度

LCS 也就是最长公共子序列，暴力肯定不是我们想要的，那么动态规划最重要的就是状态转移方程。

$$f[i][j] = \begin{cases} f[i-1][j-1] + 1 & s1[i] = s2[j] \\ \min(f[i-1][j], f[i][j-1]) & s1[i] \neq s2[j] \end{cases}$$

$f[i][j]$ 代表的是 $s1$ 的前 i 个和 $s2$ 的前 j 个的最长公共子序列的长度，不难解释，如果当前相等的话（ $s1[i]$ 与 $s2[j]$ ），那么就可以在之前（ $f[i-1][j-1]$ ）的基础上 +1，如果不相等的话就从之前的最优解（ $f[i-1][j]$ 与 $f[i][j-1]$ ）转移过来

```
1. s1=" "+s1;s2=" "+s2; //想要从 1 开始，前面统一加上空格
2. for (int i=1;i<=l1;i++)
3.     for (int j=1;j<=l2;j++)
4.         if (s1[i]==s2[j]) f[i][j]=f[i-1][j-1]+1;
5.         else
6.             f[i][j]=min(f[i-1][j],f[i][j-1]);
7. cout<<f[l1][l2]<<endl;
```

2.2.2 输出路径

我们用一个`steps`数组来存当前状态是从哪转移过来的，我这里用1、2、3分别定义三种方向，1是从左边 $\leftarrow$ （也就是 $f[i][j-1]$ ），2是从左上 $\nwarrow$ （也就是 $f[i-1][j-1]$ ），3是从上边 $\uparrow$ （也就是 $f[i-1][j]$ ）。然后从后往前去递归寻找输出。

```
1. int main()
2. {
3.     cin>>s1>>s2;
4.     l1=s1.length();l2=s2.length();
5.     s1=" "+s1;s2=" "+s2;
6.     for (int i=1;i<=l1;i++)
7.     {
8.         for (int j=1;j<=l2;j++)
9.         {
10.            if (s1[i]==s2[j]) f[i][j]=f[i-1][j-1]+1,steps[i][j]=2;
11.            else
12.            {
13.                if (f[i-1][j]>f[i][j-1]) f[i][j]=f[i-1][j],steps[i][j]=3;
14.                else f[i][j]=f[i][j-1],steps[i][j]=1;
15.            }
16.        }
17.    }
18.    //steps[i][j]存的是三种方向
19.    cout<<f[l1][l2]<<endl;
20.    LCS(l1,l2);
21.    //递归寻找
22.    return 0;
23. }
24. inline void LCS(int x,int y)
25. {
26.     //3 种状态
27.     if (steps[x][y]==2)
28.     {
29.         LCS(x-1,y-1);
30.         cout<<s2[y];
31.         //cout<<s1[x];也是一样的
32.     }
33.     if (steps[x][y]==1) LCS(x,y-1);
34.     if (steps[x][y]==3) LCS(x-1,y);
35.     //递归寻找
36.     return;
37. }
```

当我们全部搜完之后再返回输出，就是正确的顺序。

2.3 最长不下降/上升子序列(LIS)

2.3.1 输出长度

LIS 即最长上升（不下降）子序列，最主要的是一个 d 数组， $d[i]$ 表示的是长度为 i ，也就是从 1 到 i ，第 i 位最小的值。

比如一个 a 数组 = {3, 2, 1, 2}，那么从前往后扫，一开始 $d[1] = 3$ ，因为长度为 1 的话，3 是满足最小的条件，当指到 2 时候，进行维护， $d[1] = 2$ 。也就是说，长度为 1 的话，最小的是 2 就可以满足了，同理，当 $a[i] = 1$ 时候， $d[1]$ 更新为 1，因为长度为一，最后那一位是 1 即可满足。为什么会这样呢，接下来看下一位的 2，它比 $d[1]$ 要大，那么长度就 +1， $d[2] = 2$ 。

这个 $d[2] = 2$ 的意思就是，长度为 2 的最长不下降/上升子序列，最后一位是 2 即可满足，要是后面如果还有 5 的话，那么就会在 2 的基础上 $len++$ ， $d[3] = 5$ ；

我们易知， d 数组是单调递增的，只有递增才能使得 $len++$ ，不然就会替换最优解

```
1. cin>>n;
2. for (int i=1;i<=n;i++) cin>>a[i];
3. d[1]=a[1];len=1;
4. for (int i=2;i<=n;i++)
5. {
6.     if (a[i]>d[len]) d[++len]=a[i];
7.     else
8.     {
9.         int t=upper_bound(d+1,d+1+len,a[i])-d;
10.        //因为 d 是单调递增的，所以二分查找位置 t
11.        //最长上升子序列 不下降用 lower_bound
12.        d[t]=a[i];
13.    }
14. }
15. cout<<len;
```

这样的时间复杂度是 $O(n\log n)$ 。

2.3.2 输出路径

如果是想要输出路径，我们需要将每一次的 $d[i]$ 记录下来，只要有一次 $d[i]$ 赋值（而且只有一次变化），都用一个 $pos[i]$ 记录下来，如有一个数组 $a = \{8, 3, 4, 2, 6, 5, 6\}$ ，那么经过循环得到 $pos[i] = \{1, 1, 2, 1, 3, 2, 4\}$ ，这样我们从后往前，遇到的第一个当前的就记录到 ans 下，如 $\{1, 1, 2, 1, 3, 2, 4\}$ 中标粗的就是我们需要的答案，为什么倒数第二个的 2 没有用？那是因为你的 3 还没出现，3 是无法继承倒数第二个的 2 的。

所以我们用 pos 来存，从后往前找，如果找到了当前所需要的，那么才是我们的答案。

```
1. cin>>n;
2. for (int i=1;i<=n;i++) cin>>a[i];
3. d[1]=a[1];len=1;pos[1]=1;
4. for (int i=2;i<=n;i++)
5. {
6.     if (a[i]>d[len]) d[++len]=a[i],pos[i]=len;
7.     else
```

```

8.     {
9.         int t=upper_bound(d+1,d+1+len,a[i])-d;
10.        //d 单调递增
11.        //最长上升子序列 不下降用 lower_bound
12.        d[t]=a[i];
13.        pos[i]=t;
14.    }
15. }
16. cout<<len<<endl;
17. int tmp=len,cnt=n;//tmp 就是所说的当前的
18. while (tmp)
19. {
20.     while (pos[cnt]!=tmp) cnt--;
21.     ans[tmp]=a[cnt];
22.     tmp--;
23. }
24. for (int i=1;i<=len;i++) cout<<ans[i]<<" ";

```

2.4 最长公共子串

字串要求连续，那么如果不相等就不能继承转移，也就是 LCS 中的 $f[i][j] = 0$ （如果不相等的话）。

代码与 LCS 类似，只是无法继承部分情况。

```

1. cin>>s1>>s2;
2. l1=s1.length();l2=s2.length();
3. s1=" "+s1;s2=" "+s2;
4. for (int i=1;i<=l1;i++)
5. {
6.     for (int j=1;j<=l2;j++)
7.     {
8.         if (s1[i]==s2[j])
9.             f[i][j]=f[i-1][j-1]+1,ans=max(ans,f[i][j]);
10.        else
11.            f[i][j]=0;
12.    }
13. }
14. cout<<ans<<endl;
15. return 0;

```

2.5 最长回文子串

我们将字符串翻转，然后求最长公共子串，例如 $s = \text{"cbbd"}$ ，我们反转得到 $st = \text{"dbbc"}$ ，

那么最长公共子串就是 `ans=“bb”`，但是这个方法在 `leetcode` 里面的最长回文子串会超时最后一个点。

我们采用动态规划，设置一层循环模拟长度 l

```
1. cin>>st;
2. len=st.length();
3. if (len==0||len==1) {cout<<st;return 0;}
4. maxn=1;
5. for (int i=0;i<len;i++)
6. {
7.     f[i][i]=1;
8.     if (i+1<len&&st[i]==st[i+1]) f[i][i]=2,maxn=2,s=i;
9. }
10. //预处理，将长度为1和2的处理
11. for (int l=3;l<=len;l++)
12. //模拟长度
13. {
14.     for (int i=0;i+l-1<len;i++)
15.     {
16.         int j=i+l-1;
17.         if (st[i]==st[j]&&f[i+1][j-1]==1)
18.             f[i][j]=1,s=i,maxn=l;
19.     }
20. }
21. cout<<st.substr(s,maxn);
22. return 0;
```

2.6 Manacher 算法

Manacher 算法也就是我们俗称的“马拉车”算法，Manacher 算法是一个用来查找一个字符串中的最长回文子串(不是最长回文序列)的线性算法。它的优点就是把时间复杂度为 $O(n^2)$ 的暴力算法优化到了 $O(n)$ 。

它很好的处理了奇偶回文的情况，再首位增加一个不会用到的特殊符号，如‘&’与‘@’，再在中间增加一个无法用到的符号，通常用‘#’来表示，这样得到了一个新的字符串。

```
1. void init()
2. {
3.     str[0]='!';
4.     str[1]='#';
5.     for (int i=0;i<len1;i++)
6.         str[i*2+2]=s[i],str[i*2+3]='#';
7.     len2=len1*2+2;
8.     str[len2]='*';
9. }
```

$p[i]$ 数组代表以 i 为中心的回文半径， mx 就是以这个回文串为半径所能延展的最远的地方，回文中心就是 id 。回文半径和回文直径：因为处理后回文字符串的长度一定是奇数，所以回文半径是包括回文中心在内的回文子串的一半的长度，回文直径则是回文半径的2倍减1。比如对于字符串 "aba"，在字符 'b' 处的回文半径就是2，回文直径就是3。

```
1. void manacher()  
2. {  
3.     int id=0,mx=0;  
4.     for (int i=1;i<len2;i++)  
5.     {  
6.         if (mx>1) p[i]=min(p[2*id-1],mx-i);  
7.         else p[i]=1;  
8.         for (;str[i+p[i]]==str[i-p[i]];p[i]++);  
9.         if (p[i]+i>mx)  
10.             mx=p[i]+i,id=i;  
11.     }  
12. }
```

3 数据结构

3.1 并查集

并查集（Union-find Sets）是一种非常精巧而实用的数据结构，它主要用于处理一些不相交集合并的问题。一些常见的用途有求连通子图、求最小生成树的 Kruskal 算法和求最近公共祖先（Least Common Ancestors, LCA）等。

设置一个 `father` 数组来记录 i 的父亲节点，这样可以类似于将两个点连起来，通过路径压缩，可以将某一个集合深度只有2，只有一个父节点，父节点下面连上若干个子节点，这样形成一个集合，查找的时候深度不大时间复杂度很小。

一开始所有的元素都是一个个个体，所以它的父节点是自己。

```
1. for (int i=1;i<=n;i++)  
2.     father[i]=i;
```

找最上面的父节点的时候，我们可以边找边压缩，这就是并查集里面的路径压缩。

```
1. int find(int x)//查找根节点  
2. {  
3.     return father[x]==x?x:find(father[x]);  
4. }
```

最后合并的时候，先判断二者的父节点是不是一样的，也就是是否属于同一个集合的，若不属于，则将它们合并。

```
1. void merge(int x,int y)//推断 x,y 是否连通。若不连通。则使其连通。  
2. {  
3.     int fx,fy;
```

```

4.     fx=find(x);
5.     fy=find(y);
6.     if(fx!=fy) father[fx]=fy;
7. }

```

3.2 树状数组

树状数组 (Binary Indexed Tree), 又称二叉索引树, 是一个查询和修改复杂度都为 $\log(n)$ 的数据结构。主要用于数组的单点或区间修改还有区间求和。

树状数组精髓就是在于这个 *lowbit*, 它控制了每个区域如何调度长度。通过二进制的计算, 我们用 $x \& (-x)$ 用 *lowbit* 来简写。

```

1. int n;
2. int a[1005],c[1005]; //对应原数组和树状数组
3.
4. int lowbit(int x){
5.     return x&(-x);
6. }
7.
8. void updata(int i,int k){ //在 i 位置加上 k
9.     while(i <= n){
10.         c[i] += k;
11.         i += lowbit(i);
12.     }
13. }
14.
15. int getsum(int i){ //求 A[1 - i] 的和
16.     int res = 0;
17.     while(i > 0){
18.         res += c[i];
19.         i -= lowbit(i);
20.     }
21.     return res;
22. }

```

树状数组我用的不多, 树状数组还可以求逆序对, 逆序对的常用做法是归并排序, 这样树状数组可能会简单一些。

对于一般区间的问题, 我一般用强大的线段树, 但是线段树又有点写起来麻烦, 有时候会用分块来写, 分块也有点麻烦。

3.3 线段树

3.3.1 线段树简介

线段树 (Segment Tree) 是一种二叉搜索树, 将一个区间分成一些单元区间, 一些单元区间由一个节点来掌控。线段树适用于一些 “区间加法” 问题。

3.3.2 区间加法条件

何谓“区间加法”？举例如下。

区间求和——一个区间和=左区间和+右区间和

最大公因数(GCD)——总区间 $GCD = gcd(\text{左区间 } GCD, \text{右区间 } GCD)$;

最大值——总最大值= $\max(\text{左区间最大值}, \text{右区间最大值})$

有一些不符合“区间加法的”，例如：众数，左区间众数与右区间众数不一样，无法求得总区间众数。

3.3.3 线段树操作

满足“区间加法”的条件，我们就可以使用线段树，对于我们每一次更新，我们都从根节点往下，满足在 $[L, R]$ 的区间我们就进行，其实也就是将 $[L, R]$ 区间分成很多小区间，更新完之后我们再回溯上来更新控制区间的值，一直回到根节点。

对于每一次查询也是同样的道理，将 $[L, R]$ 区间分成很多小区间，并且一直往下，最后得到返回值，因为是一个树形结构，每一次的修改和查询的时间复杂度是 $O(\log n)$ 的。

3.3.4 线段树优化（常数优化+pushdown 优化）

虽然线段树是很快，但是我们还是可以优化，有一个最重要的就是 lazy 标记操作，我们对于每一次修改，我们如果不用就没必要再往下更新，如果以后都没有查询这个区间，那么我们省下了一笔不错的时间；如果以后用到了这个区间，我们就用 pushdown 将其之前操作一起往下，这样如果很多 lazy 都积累在一起，然后一起 pushdown 的话，这个时间复杂度是大大减少的。这个 pushdown 和 lazy 操作在分块中也经常出现。

对于常数优化，我们可以用位运算

```
1. #define lson (o<<1)
2. #define rson (o<<1|1)
3. #define mid ((l+r)>>1)
```

3.3.5 封装好的线段树模板代码

```
1. struct Segment_Tree
2. {
3.     LL sumv[N<<2], addv[N<<2], lc[N<<2], rc[N<<2];
4.     #define lson (o<<1)
5.     #define rson (o<<1|1)
6.     #define mid ((l+r)>>1)
7.     inline void build(LL o, LL l, LL r)
8.     {
9.         lc[o]=l; rc[o]=r;
10.         if (l==r) {sumv[o]=a[l]; return ;}
11.         build(lson, l, mid); build(rson, mid+1, r);
12.         pushup(o);
13.     }
14.     inline void pushup(LL o)
15.     {
16.         sumv[o]=sumv[lson]+sumv[rson];
17.     }
```

```

18.     inline void pushdown(LL o)
19.     {
20.         if (addv[o])
21.         {
22.             sumv[lson]+=addv[o]*(rc[lson]-lc[lson]+1);
23.             sumv[rson]+=addv[o]*(rc[rson]-lc[rson]+1);
24.             addv[lson]+=addv[o];
25.             addv[rson]+=addv[o];
26.             addv[o]=0;
27.         }
28.     }
29.     inline void optadd(LL o,LL l,LL r,LL ql,LL qr,LL k)
30.     {
31.         if (ql<=l&&r<=qr)
32.         {
33.             sumv[o]+=k*(rc[o]-lc[o]+1);
34.             addv[o]+=k;
35.             return;
36.         }
37.         pushdown(o);
38.         if (ql<=mid) optadd(lson,l,mid,ql,qr,k);
39.         if (qr>mid) optadd(rson,mid+1,r,ql,qr,k);
40.         pushup(o);
41.     }
42.     inline LL query(LL o,LL l,LL r,LL ql,LL qr)
43.     {
44.         if (ql<=l&&r<=qr) return sumv[o];
45.         pushdown(o);
46.         LL ans=0;
47.         if (ql<=mid) ans+=query(lson,l,mid,ql,qr);
48.         if (qr>mid) ans+=query(rson,mid+1,r,ql,qr);
49.         return ans;
50.     }
51. }tree;

```

3.4 ST 表

3.4.1 ST 表介绍

ST 表 (Sparse Table) 又称稀疏表, 是一种简单的数据结构, 主要用来解决 RMQ (Range Maximum/Minimum Query, 区间最大/最小值查询) 问题。它主要应用倍增的思想, 可以实现 $O(n \log n)$ 预处理、 $O(1)$ 查询。

除了解决求最大值/最小值之外, 常用于解决可重复贡献问题, 也就是上述线段树的“区间加法”, 那么有以下几个问题类型属于可重复贡献: 区间最值、区间接位和、区间接位或、区间 GCD 等。而像区间和这样的问题就不是可重复贡献问题。

ST 表往往对于降时间复杂度有奇效。

3.4.2 ST 表构建

ST 表是基于倍增思想，我们定义 $f[i][j]$ 表示区间 $[i, i + 2^j - 1]$ 内的最值，最初 $f[i][0] = \max\{i, i\} = a[i]$ 。所以 $f[i][j] = \max(f[i][j-1], f[i + 2^{j-1}][j-1])$ 。

```
1. for (int i=n;i>=1;i--)
2.     for (int j=1;i+(1<<j)-1<=n;j++)
3.         f[i][j]=Min(f[i][j-1],f[i+(1<<j-1)][j-1]);
```

3.4.3 ST 表查询

我们将 $[l, r]$ 分为两个区间，一部分是从左边开始向右边伸长 2^k 的，一部分是右边开始往左边伸长 2^k 的，具体 k 怎么来满足使得双方的交集都在 $[l, r]$ 内呢？这个 k 就是区间长度向下取整得到的， $k = \log_2(r - l + 1)$ 。

```
1. for (int i=1;i<=m;i++)
2. {
3.     scanf("%d%d",&l,&r);
4.     k=lg2[r-l+1];
5.     printf("%d\n",Min(f[l][k],f[r-(1<<k)+1][k]));
6. }
```

3.4.4 ST 表预处理

可以预处理出来 $lg2$

```
1. for (int i=2;i<=n;i++)
2.     lg2[i]=lg2[i>>1]+1;
```

3.5 分块

3.5.1 分块简介

分块是将一个整体分成若干个小块，处理区间问题的时候对于整块就整块地处理，对于不完整块就暴力处理，是一种处理区间问题的数据结构。分块算法也因此被称为“优雅的暴力”。不难证明，时间复杂度在 $O(n\sqrt{n})$ 的算法，比 $O(n\log n)$ 级别的稍慢一些。

分块算法相较于各种树形数据结构，具有简便易写，方便调试等多种优点。在同等数据规模下，如 $1e5$ ，其时间效率并不会低太多，在考试时反而是一种有力的得分方法。有些题目比如求区间众数，或者求区间小于 k 的数有多少往往不是线段树能解决的。

分块入门的话有非常出名的分块 [hzwer 入门 1-9](#)，前段时间认认真真将分块九讲完完全全做了一遍，并且专门开了一个分栏写在了博客之中。莫队算法也是基于分块的。

这一路做下来，我觉得分块一般分成三个部分：1) 预处理；2) 更新操作；3) 询问。有些时候的询问和操作是放在一起的，比如入门 8 的查询之后的赋值。

博客链接 https://blog.csdn.net/Carry_hkr/article/details/118600183

3.5.2 预处理

预处理一般处理一些 $block$ (块的长度), num (块的个数), $l[i]$ (块 i 的最左边的位置), $r[i]$ (块 i 的最右边位置), 还有一个 $belong[i]$ (表示第 i 个位置是属于哪一块的)。常规的是这些，还有具

体情况具体分析的，比如初始化 $flag$ 标志数组为 -1 (如入门 8)等。有些题解是没有写 l, r 数组的，因为可以直接表示出来，我觉得那样代码虽然短，但是不美观，有时候不太好理解，这样预处理之后可以一劳永逸了。

这里注意，因为 n 往往不是完全平方数，所以我们需要将右边多出的不完整块来重新分配成一个块。

```
1. void build()
2. {
3.     block=sqrt(n);
4.     num=n/block;
5.     if (n%block) num++;
6.     for (int i=1;i<=num;i++)
7.         l[i]=(i-1)*block+1,r[i]=i*block;
8.     r[num]=n;
9.     for (int i=1;i<=n;i++)
10.        belong[i]=(i-1)/block+1;
11.    for (int i=1;i<=num;i++)
12.        for (int j=l[i];j<=r[i];j++)
13.            Max[i]=max(Max[i],a[j]);
14.    //这里拿区间最大值来举例
15. }
```

3.5.3 更新操作

操作的话大部分是以区间操作进行，除了一些特殊的，比如入门 6 的单点插入，单点查询，区间操作分块就起大作用了。块，分为两种：**完整块**，我们一般用 $lazy$ 等懒惰处理，需要用的时候再去进行更新等操作；**不完整块**，我们在类似 $lazy$ 的基础上进行，有时候需要先全部更新一下，例如入门 8 的 $flag$ 和 $reset$ ，然后再暴力修改。还有一个重要的地方，如果对于区间比较小，也就是说 $belong[l] = belong[r]$ ，不完整块只有一块，不需要从右边界再处理一次不完整块。

```
1. inline void update(int x,int y,int c)
2. {
3.     if (belong[x]==belong[y])
4.     {
5.         for (int i=x;i<=y;i++) a[i]+=c;
6.         return;
7.     }
8.     for (int i=x;i<=r[belong[x]];i++) a[i]+=c;
9.     for (int i=belong[x]+1;i<belong[y];i++) lazy[i]+=c;
10.    for (int i=l[belong[y]];i<=y;i++) a[i]+=c;
11.    //这个用区间加 c 来举例
12. }
```

3.5.4 查询

查询其实也跟操作一样，对于不完整块我们暴力查询，对于完整块我们就用类似于 $lazy$

操作得到答案，比如入门 9 的 $maxn[i][j]$ 能直接得到第 i 块到第 j 块的最小众数。我们也需要注意一下左右区间边界是否是属于同一块的情况下，避免重复。

3.5.5 其他注意事项

这些题目总结起来的特殊情况。

- 1) 依情况而定，是否需要取余
- 2) 是否需要开 long long
- 3) 对于一个区间排序二分，赋值的时候需要整体将整块赋值给排序数组，然后排序
- 4) 有些题目 $block$ 不易太小，可以手动设置 $block = 200$ 等，不然会 t

3.5.6 分块优化

具体见 3.6 莫队的优化，有些题目就是为了卡莫队，为了让你用主席树等数据结构。

3.6 莫队算法

3.6.1 莫队算法介绍

莫队，莫涛，长郡中学学生，算我半个学长了，后进入清华大学学习，前 IOI 中国国家队长发明了“莫队算法”，在国外名字叫做"Mo's Algorithm"。

莫队算法的精髓就是通过合理地对询问排序，然后以较优的顺序暴力回答每个询问。处理完一个询问后，可以使用它的信息得到下一个询问区间的答案。莫队是一种离线的算法。

3.6.2 按照较优的顺序

我们设置一个结构体，存储我们需要的变量，有 l, r, id ，可能还有其他的。

```
1. struct Mo{int l,r,ID,k;int ans;}q[N];
```

那么这个较优的顺序暴力是如何做到的呢？这里运用分块的思想，如果是属于同一块的话，那么比较左区间；如果不是同一块的话就比较右区间。这样保证左右指针在每一次与上一次询问移动的不多。如果不排序，像是 $[1,2]$ 和 $[99999,100000]$ 这样的数据那会花费大量的时间移动指针。

```
1. bool cmp(Mo a,Mo b)
2. {
3.     return belong[a.l]==belong[b.l]?a.r<b.r:a.l<b.l;
4. }
```

3.6.3 询问

对于处理下一个询问，莫队喜欢用 while 循环移动，判断 l, r 两个指针的移动方向，直到指针与区间相等。

注意初始设置 $l = 1, r = 0$ 。

```
1. l=1;r=0;
2. for (int i=1;i<=m;i++)
3. {
4.     while(l<q[i].l) del(l++);
5.     while(l>q[i].l) add(--l);
6.     while(r<q[i].r) add(++r);
7.     while(r>q[i].r) del(r--);
```

```

8.     q[i].ans=...
9. }

```

然后每一次处理对于移动前要**减去**，然后**处理**，最后将处理完的**加上来**，这样保证答案对于当前的 l, r 是正确的，那么到最后的区间也是正确的。

```

1. void add(int x)
2. {
3.     num[sum[a[x]]]--;
4.     sum[a[x]]++;
5.     num[sum[a[x]]]++;
6. }
7. void del(int x)
8. {
9.     num[sum[a[x]]]--;
10.    sum[a[x]]--;
11.    num[sum[a[x]]]++;
12. }
13. //num 和 sum 举个例子

```

最后按照原来的顺序输出即可。

3.6.4 奇偶排序优化

对于莫队的分块思想，我们进行了 sort 排序，同一块的放一起，实际上还能细分，也就是奇偶排序。

将原本的 cmp 换成奇偶排序，并且用到位运算省时间。

```

1. int cmp(query a, query b)
2. {
3.     return (belong[a.l] ^ belong[b.l]) ? belong[a.l] < belong[b.l] : ((belong[a.l] & 1) ? a.r < b.r : a.r > b.r);
4. }

```

也就是说，对于左端点在同一奇数块的区间，右端点按升序排列，反之降序。这个东西也是看着没用，但实际效果显著。

它的主要原理便是右指针跳完奇数块往回跳时在同一个方向能顺路把偶数块跳完，然后跳完这个偶数块又能顺带把下一个奇数块跳完。理论上主算法运行时间减半，实际情况有所偏差。

3.6.5 常数优化

在一些细节上面进行优化，比如减少函数调用，while 循环一句话搞定。比如：

```

1. for (register int i=1;i<=m;i++)
2. {
3.     while(l<q[i].l) now -= !--cnt[a[l++]];
4.     while(l>q[i].l) now += !cnt[a[--l]]++;

```

```

5.     while(r<q[i].r) now += !cnt[a[++r]]++;
6.     while(r>q[i].r) now -= !--cnt[a[r--]];
7.     //q[i].ans=now;
8.     answer[q[i].ID]=now;
9.     //最后不进行 sort 直接按照 answer 输出
10. }

```

还可以最后用`answer`数组来存储答案，不需要排序后输出，我们还可以快读快输进行优化等。

总而言之，莫队是一种 $O(n\sqrt{n})$ 的算法，比 $O(n\log n)$ 的算法在数据很大的情况下还是有些差距的，这时候我们考虑换数据结构来写吧，毕竟这样降常数经不起折腾，但是莫队还是有很多问题是线段树等解决不了的。

4 图论

4.1 图论综述

4.1.1 图的定义

图论 (Graph theory) 是数学的一个分支，图是图论的主要研究对象。图 (Graph) 是由若干给定的顶点及连接两顶点的边所构成的图形，这种图形通常用来描述某些事物之间的某种特定关系。顶点用于代表事物，连接两顶点的边则用于表示两个事物间具有这种关系。

4.1.2 邻接矩阵建图

用一个二维数组 $f[i][j]$ 来表示节点 i 到节点 j 有一条有向边，如果是无向边那么就加上反向 $f[j][i] = 1$ 。遍历的时候就枚举所有情况，判断是否值为1，如果为1那么就是有向边。

4.1.3 链式前向星邻接表建图

这个是我习惯的建图方式，是链表结构的变种邻接表。

```

1. struct node
2. {
3.     int to,next,w;
4. }edge[N<<1];
5. inline void add(int x,int to,int w)
6. {
7.     edge[++cnt].to=to;
8.     edge[cnt].next=head[x];
9.     edge[cnt].w=w;
10.    head[x]=cnt;
11. }

```

遍历的时候利用到这个 `head[x]`开始。

```

1. for (int i=head[x];i;i=edge[i].next)
2. {
3.     int y=edge[i].to,w=edge[i].w;
4.     if (exist[y]) continue;

```

```

5.     if (...)
6.     {
7.         ...
8.         dfs(y);
9.         ...
10.    }
11. }

```

4.1.4 vector 邻接表建图

Vector 建图就是使用可以动态开点的数据结构，比如 **vector** 数组，通过动态开点节省空间，同时对遍历等操作复杂度也很友好，但相对的，查询某条边是否存在的复杂度相对邻接矩阵则变大了。

```

1.  for (int i=1;i<=m;i++)
2.  {
3.      cin>>u>>v>>w;
4.      G[u].push_back({v,w});
5.  }
6.  ...
7.  for (auto i:G)
8.  {
9.      int v=i.first,w=i.second;
10.     ...
11. }

```

4.2 拓扑排序

拓扑排序 (Topological Sorting) 是要解决的问题是给一个图的所有节点排序。当这个节点之前的所有点都在前面解决之后才能轮到该节点，这样的顺序可以通过拓扑排序解决。

我们可以说在一个 DAG (有向无环图) 中，我们将图中的顶点以线性方式进行排序，使得对于任何的顶点 u 到 v 的有向边 (u, v) ，都可以有 u 在 v 的前面。

还有给定一个 DAG，如果从 i 到 j 有边，则认为 j 依赖于 i ，如果 i 到 j 有路径 (i 可达 j)，则称 j 间接依赖于 i 。

拓扑排序的目标是将所有节点排序，使得排在前面的节点不能依赖于排在后面的节点。

```

1.  for (int i=1;i<=m;i++)
2.  {
3.      cin>>u>>v;
4.      add(u,v);
5.      in[v]++;
6.      //统计入度
7.  }
8.  ...

```

```

9. vector<int> tp;
10. int topo_cnt[N];
11. inline void topo()
12. {
13.     queue<int> Q;
14.     int t=0;
15.     for (int i=1;i<=n;i++)
16.         if (!in[i])
17.         {
18.             Q.push(i);
19.             topo_cnt[i]=++t;
20.             tp.push_back(i);
21.         }
22.     while (!Q.empty())
23.     {
24.         int now=Q.front();Q.pop();
25.         for (int i=head[now];i;i=edge[i].next)
26.         {
27.             int y=edge[i].to;
28.             in[y]--;
29.             if (!in[y])
30.             {
31.                 Q.push(y);
32.                 topo_cnt[y]=++t;
33.                 tp.push_back(y);
34.             }
35.         }
36.     }
37.     if (t!=n) cout<<"It is not DAG!";
38.     else
39.         for (auto i:tp) cout<<i<<" ";
40.     cout<<endl;
41. }

```

这个拓扑排序对于 1.2.3 那道记忆化搜索也是同样适用的。

4.3 最小生成树

4.3.1 最小生成树简介

最小生成树是一副连通加权无向图中一棵权值最小的生成树。

它在实际中有什么应用呢？比如说有 N 个城市需要建立互联的通信网路，如何使得需要铺设的通信电缆的总长度最小呢？这就需要用到最小生成树的思想了。

4.3.2 Prim 算法

Prim 算法是贪心算法，贪心策略为：找到目前情况下能连上的权值最小的边的另一端点，加入之，直到所有的顶点加入完毕。

Prim 适用于稠密图。

朴素 Prim 的时间复杂度是 $O(n^2)$ 。因为在寻找离生成树最近的未加入顶点时浪费了很多时间，所以可以用堆进行优化。堆优化后的 Prim 算法的时间复杂度为 $O(m \log n)$ 。不过堆优化 Prim 的代码比较复杂，而并查集优化的 Kruskal 算法与它相比要好很多。

Prim 算法原理：

- 1) 以某一个点开始，寻找当前该点可以访问的所有的边；
- 2) 在已经寻找的边中发现最小边，这个边必须有一个点还没有访问过，将还没有访问的点加入我们的集合，记录添加的边；
- 3) 寻找当前集合可以访问的所有边，重复 2 的过程，直到没有新的点可以加入；
- 4) 此时由所有边构成的树即为最小生成树。

因为代码复杂和个人习惯，如果不是必要要用到 Prim 算法，我个人一般习惯用 Kruskal 算法来处理最小生成树问题。

4.3.3 Kruskal 算法

Kruskal 算法也是贪心的算法，我们对象是边，我们不像存图那样存每个点与点之间连边，我们直接设置一个结构体分别有 u, v, w ，对于边权 w 我们进行从小到大的排序。因为一个点总数为 n 的话，我们连成一棵树需要 $n - 1$ 条边，我们设置一个计数变量，待到增加到 $n - 1$ 的时候即可推出。对于每一条边，我们先判断是否属于一个集合里面，也就是利用了并查集的方法，如果在一个集合里面，虽然边权目前来说最小，但也没有用。

```
1. void Kruskal()
2. {
3.     for (int i=1;i<=n;i++)
4.         father[i]=i;
5.     sort(a+1,a+1+m,cmp);
6.     for (int i=1;i<=m;i++)
7.     {
8.         fa=find(a[i].x);fb=find(a[i].y);
9.         if (fa!=fb)
10.        {
11.            father[fb]=fa;
12.            tot+=a[i].v;
13.            k++;
14.        }
15.        if (k==n-1) break;
16.    }
17.    if (k==n-1) cout<<tot;
18.    else cout<<-1;
19. }
```

4.3.4 次小生成树(MIT)

次小生成树就是另外一颗树比当前生成的最小生成树情况下的总权值要大于等于它，大于的话是严格次小生成树，需要用到最近公共祖先(LCA)+倍增思想。

对于次小生成树，稍加思考，我们可以想到，次小生成树与最小生成树差的只是一条边。

为什么呢？我们先建出一棵最小生成树，满足使用的边都是最小的，剩下的边（称为非树边）一定没有树边优。如果我们加入一条非树边，删除最小生成树中的一条边，次小生成树一定是包括在以这种方法建出的树中的（倘若删两条树边加两条非树边，则肯定没有删一

条加一条边，绝不是次小生成树)

思想就是枚举不在最小生成树的边，加入最小生成树后一定会形成环路，删除掉环路中最大的边，比较答案即可。

4.4 二分图匹配

4.4.1 二分图定义

二分图：有两顶点集且图中每条边的两个顶点分别位于两个顶点集中，每个顶点集中没有边直接相连接。

简单的说，一个图被分成了两部分，相同的部分没有边，那这个图就是二分图，二分图是特殊的图。

4.4.2 二分图的判定

如果某个图为二分图，那么它至少有两个顶点，且其所有回路的长度均为偶数，任何无回路的图均是二分图。

判断二分图的常见方法是染色法：用两种颜色，对所有顶点逐个染色，且相邻顶点染不同的颜色，如果发现相邻顶点染了同一种颜色，就认为此图不为二分图。当所有顶点都被染色，且没有发现同色的相邻顶点，就退出。

```
1. bool dfs(int v, int c)
2. {
3.     color[v] = c;    //将当前顶点涂色
4.     for(int i = 0; i < n; i++)
5.     {    //遍历所有相邻顶点，即连着的点
6.         if(edge[v][i] == 1)
7.         {    //如果顶点存在
8.             if(color[i] == c)    //如果颜色重复，就返回 false
9.                 return false;
10.            if(color[i] == 0 && !dfs(i, -c))
11.                //如果还未涂色，就染上相反的颜色-c,并 dfs 这个顶点，进入下一层
12.                return false;    //返回 false
13.        }
14.    }
15.    return true;    //如果所有顶点涂完色，并且没有出现同色的相邻顶点，就返回 true
16. }
17. void solve()
18. {
19.     for(int i = 0; i < n; i++)
20.     {
21.         if(color[i] == 0)
22.         {
23.             if(!dfs(i, 1))
24.             {
25.                 printf("NOT BICOLORABLE.\n");
26.                 return;
27.             }
28.         }
```

```

29.     }
30.     printf("BICOLORABLE.\n");
31. }

```

4.4.3 匈牙利算法

匈牙利算法是二分图匹配的核心算法，除了二分图其他多重匹配外均可使用，实际上就是一种网络流的思想，核心就是寻找增广路。

匹配就是一个递归的过程，如果当前没匹配那么就匹配，然后继续下一个；如果当前匹配了，那么试着让之前匹配的改变匹配对象。

举个例子，我们现在有一部分男人和女人他们各自对立寻找可能对象(一条边)，男 1 去寻找他可能潜在的对象(有边相连)，假如连上了女 1，那么就改轮到男 2 去找。很不巧女 1 也是男 2 的潜在对象(男 2 与女 1 之间有边)，那么就让女 1 与男 2 相匹配，让男 1 重新去找...如此循环递归。

```

1.  inline bool Hdfs(int x)
2.  {
3.      for (int i=head[x];i=edge[i].next)
4.      {
5.          int y=edge[i].to;
6.          if (chw[y])
7.          {
8.              chw[y]=false;
9.              if (!match[y] || Hdfs(match[y]))
10.             {
11.                 match[y]=x;
12.                 return 1;
13.             }
14.         }
15.     }
16.     return 0;
17. }
18. int main()
19. {
20.     ...
21.     for (int i=1;i<=n;i++)
22.     {
23.         memset(chw,1,sizeof(chw));
24.         if (Hdfs(i)) ans++;
25.     }
26.     ...
27. }

```

4.5 最短路问题

4.5.1 最短路简介

顾名思义，是一张图里面，两个点之间的不同路径中，全值最短的那一条的值。

4.5.2 最短路方法

最短路的选择一般有四种：Floyd 算法、Dijkstra 算法、SPFA 算法、Bellman-Ford 算法。它们各自有各自的优点。

Floyd 是多源最短路算法，而另外几种是单源最短路算法。Floyd 可以一次性计算任意两个点之间的最短路，在处理多源最短路时它几乎是唯一的选择。它的时间复杂度是 $O(n^3)$ ，这使得它只能处理很小的一部分数据。而因为使用 Floyd 时节点数不会很多，所以可以选择使用更易使用的邻接矩阵存图。

Dijkstra 只适用于无负权图，经过优化的 Dijkstra 算法可以说是非负权图单源最短路的王者，在无负权，单源最短路，数据范围很大时，Dijkstra 几乎成了最优选择。

SPFA 算法除了最短路外，还可以判断是否有负环，当题目涉及负环时，SPFA 几乎成了唯一选择。SPFA 简单易写，时间复杂度也不错，且可以处理负边权。在数据不大的单源最短路中可以使用。

Bellman-Ford 算法注意它的松弛操作，每次松弛操作实际上是对相邻节点的访问，第 n 次松弛操作保证了所有深度为 n 的路径最短。由于图的最短路径最长不会经过超过 $|V| - 1$ 条边，所以可知 Bellman-Ford 算法所得为最短路径。

4.5.3 Floyd 算法

Floyd 适用与一张图里面点比较小的情况，因为时间复杂度在 $O(n^3)$ 级别的，但是我们可以加入一些优化使得其可以降到 $O(n^2 \log n)$ 级别，但是还是很慢。一般题目给出 $n \leq 100$ 的情况下我们可以采用这个 Floyd 算法。

我们定义 $dis[i][j]$ 表示结点 i 到结点 j 的距离，枚举中间变量 k ，来更新 $dis[i][j]$

```
1. for (int k=1;k<=n;k++)
2.     for (int i=1;i<=n;i++)
3.         for (int j=1;j<=n;j++)
4.             if (dis[i][j]>dis[i][k]+dis[k][j])
5.                 dis[i][j]=dis[i][k]+dis[k][j];
```

4.5.4 Dijkstra 算法

注意：Dijkstra 算法只适用于所有边的权都大于0的图。

Dijkstra 算法是贪心算法。基本思想是：

设置一个顶点的集合 S ，并不断地扩充这个集合，当且仅当从源点到某个点的路径已求出时它才属于集合 S 。

开始时 S 中仅有源点，调整 S 集合之外的点的最短路径长度，并从中找到当前最短路径点，将其加入到集合 S ，直到所有的点都在 S 中。

普通算法的时间复杂度为 $O(n^2)$ 。不过还有优化的余地。

我们可以使用优先队列维护集合，每次取出最短的一条边进行操作，可以将复杂度降低到 $O(n \log n)$ 级别。

```
1. inline void Dij(int s,int t)
2. {
3.     memset(dis,0x7f/2,sizeof(dis));
4.     dis[s]=0;
5.     priority_queue<pair<int,int> > Q;
```

```

6.     Q.push({0,s});
7.     exist[s]=1;
8.     while (!Q.empty())
9.     {
10.        int x=Q.top().second;Q.pop();
11.        if (x==t) break;
12.        //如果是单源最短路 break 即可
13.        for (int i=head[x];i;i=edge[i].next)
14.        {
15.            int y=edge[i].to,w=edge[i].w;
16.            if (exist[y]) continue;
17.            if (dis[y]>dis[x]+w)
18.            {
19.                dis[y]=dis[x]+w;
20.                Q.push({-dis[y],y});
21.                exist[y]=1;
22.            }
23.        }
24.    }
25.    cout<<dis[t];
26. }

```

4.5.5 SPFA 算法

Bellman-Ford 在迭代时会有一些冗余的松弛操作。SPFA 则通过队列对其进行一个优化。

```

1.  inline void SPFA()
2.  {
3.      memset(dis,0x7f/2,sizeof(dis));
4.      dis[1]=0;
5.      Q.push(1);exist[1]=true;
6.      while (!Q.empty())
7.      {
8.          int u=Q.front();Q.pop();exist[u]=false;
9.          for (int i=head[u];i;i=edge[i].next)
10.         {
11.             int v=edge[i].to,w=edge[i].w;
12.             if (dis[v]>dis[u]+w)
13.             {
14.                 dis[v]=dis[u]+w;
15.                 if (!exist[v]) {Q.push(v);exist[v]=true;}
16.             }
17.         }
18.     }

```

```

19.     printf("%d",dis[n]);
20. }

```

4.5.6 Bellman-Ford 算法

Bellman-Ford 算法是迭代法，它不停地调整图中的顶点值（源点到该点的最短路径值），直到没有可调整的值。

该算法除了能计算最短路，还可以检查负环（一个每条边的权都小于0的环）把每条边的权取一个相反数，就可以判断图中是否有正环了。如果图中有负环，那么这个图不存在最短路。时间复杂度为 $O(mn)$ 。实践中往往使用它的优化版本 SPFA 算法，所以实现在此不表。

4.6 Tarjan 算法

4.6.1 一些概念

1)强连通定义：在有向图中，如果两个点 x,y 间存在一条 x 到 y 的路径，也存在一条 y 到 x 的路径，则称 x,y 是强连通的

2)强连通图定义：如果有向图 G 的任意两个顶点都强连通，则称 G 是一个强连通图

3)强连通分量定义：有向图的极大强连通子图，称为强连通分量

4)时间戳：在DFS过程中，根据访问的先后，用 $1 - N$ 标记 N 个节点，即为 $dfs[x]$

5)搜索树：我们对图的DFS时，是可能重复访问一个点多次的，毕竟他不是个树。我们把没处理到的边忽略，只留下我们DFS过程访问的边，就会形成一棵树。我们叫它搜索树。

6)追溯值： $low[x]$ ，我们定义 $s(x)$ 为搜索树中 x 的子树，我们定义 $low[x]$ 为 $s(x)$ ，以及通过1条边可以到达 $s(x)$ 的节点中， dfn 的最小值。

7)感性理解下， x 的时间戳就是遍历到 x 的时间， x 的追溯值就是 x 以及他的子节点们一直走下去能碰到的最小时间戳

4.6.2 Tarjan 算法

```

1. inline void Tarjan(int x)
2. {
3.     dfn[x]=low[x]=++dfs_num;
4.     exist[x]=true;stack[++top]=x;
5.     for (int i=head[x];i;i=edge[i].next)
6.     {
7.         int y=edge[i].to;
8.         if (!dfn[y])
9.         {
10.            Tarjan(y);
11.            low[x]=min(low[x],low[y]);
12.        }
13.        else if (exist[y]) low[x]=min(low[x],dfn[y]);
14.    }
15.    if (dfn[x]==low[x])
16.    {
17.        int now=0;scc++;
18.        while (x!=now)
19.        {
20.            now=stack[top--];

```

```

21.         exist[now]=0;
22.     }
23. }
24. }

```

一般一次不会找完，所以我们需要一个循环。

```

1. for (int i=1;i<=n;i++)
2.     if (!dfn[i]) Tarjan(i);

```

不难发现，Tarjan 的时间复杂度是 $O(V + E)$

Tarjan 的主要难以理解的地方就是 dfn 和 low 之间。

设 $s(x)$ 为 x 搜索树中子树，我们定义 low 是以下结点的极小时间戳。

当出现 $low[x] = dfn[x]$ 时，栈中所有点构成一个 SCC ，连通分量要求任意两点可达，而强连通分量 (SCC) 要求任意两点互相可达，即必须存在 $a \rightarrow b$ 且 $b \rightarrow a$ 的路径。

显然 $low[x]$ 不可能比 $dfn[x]$ 大，一个点永远能追溯到自己。

$low[x] = dfn[x]$ 时，就是当前栈里能回溯到的最远就是 x 了，那么取出整个栈就可以了。

如果 $low[x] < dfn[x]$ ，说明当前栈里能回溯到比 x 更远的地方，那么不能取出，接着操作。

4.6.3 割点

割点的概念：在无向连通图中，如果将其中一个点以及所有连接该点的边去掉，图就不再连通，那么这个点就叫做割点 (cut vertex / articulation point)。

利用 Tarjan 算法可以很容易求出割点。

```

1. inline void Tarjan(int u,int pre)
2. {
3.     low[u]=dfn[u]=++dfs_num;
4.     int child=0;
5.     for (int i=head[u];i;i=edge[i].next)
6.     {
7.         int v=edge[i].to;
8.         if (!dfn[v])
9.         {
10.            child++;
11.            Tarjan(v,u);
12.            low[u]=Min(low[u],low[v]);
13.            if (low[v]>=dfn[u]) iscut[u]=true;
14.        }
15.        else if (dfn[v]<dfn[u] && v!=pre)
16.            low[u]=Min(low[u],dfn[v]);
17.    }
18.    if (pre<0 && child==1) iscut[u]=0;
19. }

```

5 数论

5.1 最大公因数(GCD)

求 GCD 的可以用辗转相除法和更相减损术，一般我们采用辗转相除法，而基本思想就是 $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ ，我们可以采用递归和循环的形式，下面给出递归形式。

```
1. int gcd(int m, int n)
2. {
3.     return n ? gcd(n, m % n) : m;
4. }
```

但是如果是板子的话，我们也可以利用位运算加快速度，这个给出 while 循环形式。

```
1. int GCD(int a, int b){while(b^=a^=b^=a%=b);return a;}
```

5.2 最小公倍数(LCM)

两个数的最大公约数和最小公倍数的关系为 $\text{gcd}(a, b) \times \text{lcm}(a, b) = ab$ ，因此可以通过 $\text{lcm}(a, b) = \frac{ab}{\text{gcd}(a, b)}$ 来求出最小公倍数。

5.3 素数

5.3.1 素数定义

素数是指在大于1的自然数中，除了1和它本身以外不再有其他因数的自然数。

5.3.2 常规素数判定

判定一个数 x 是否是质数，只需要枚举它的部分因子，这部分因子指的是 $2 \sim \sqrt{x}$ ，这枚举的是一对因子里面较小的那个，这样的时间复杂度是 $O(\sqrt{n})$ ，这对于小部分数据是可以的，只要数据一大，我们就不可用这个方法。

```
1. inline bool isprime(int x)
2. {
3.     for (int i=2; i<=sqrt(x); i++)
4.         if (x%i==0) return false;
5.     return true;
6. }
```

5.3.3 线性筛素数

筛法的思想是去除要求范围内所有的合数，剩下的就是素数了，而任何合数都可以表示为素数的乘积，因此如果已知一个数为素数，则它的倍数都为合数。

每次只要筛选小于等于 i 的第一个素因子的素数与 i 的乘积，既不会造成重复筛选，又不会遗漏，时间复杂度 $O(n)$ 。

下面给出常用模板。

```

1.  memset(isprime,1,sizeof(isprime));
2.  isprime[0]=isprime[1]=false;
3.  for (int i=2;i<=n+1;i++)
4.  {
5.      if (isprime[i]) prime[++num_prime]=i;
6.      for (int j=1;j<=num_prime&& i*prime[j]<n+1;j++)
7.      {
8.          isprime[i*prime[j]]=false;
9.          if (!(i%prime[j])) break;
10.     }
11. }

```

5.3.4 Miller-Rabin 判素数算法

Miller_Rabin 算法进行素数测试，速度快可以判断一个 小于 2^{63} 的数是不是素数。随机算法判定次数一般 8 ~ 10 就够了，其时间复杂度上界为 $O(k \log n)$ ，其中 k 为检测的轮数。增大 k 可以提高 Miller-Rabin 算法的准确度。

```

1.  const int S = 8; //随机算法判定次数一般 8~10 就够了
2.  // 计算 ret = (a*b)%c a,b,c < 2^63
3.  long long mult_mod(long long a,long long b,long long c){
4.      a %= c;
5.      b %= c;
6.      long long ret = 0;
7.      long long tmp = a;
8.      while(b){
9.          if(b & 1){
10.             ret += tmp;
11.             if(ret > c)ret -= c;//直接取模慢很多
12.          }
13.          tmp <<= 1;
14.          if(tmp > c)tmp -= c;
15.          b >>= 1;
16.      }
17.      return ret;
18. }
19. // 计算 ret = (a^n)%mod
20. long long pow_mod(long long a,long long n,long long mod){
21.     long long ret = 1;
22.     long long temp = a%mod;
23.     while(n){
24.         if(n & 1)ret = mult_mod(ret,temp,mod);
25.         temp = mult_mod(temp,temp,mod);
26.         n >>= 1;
27.     }
28.     return ret;

```

```

29. }
30. // 通过  $a^{(n-1)} \equiv 1 \pmod n$  来判断 n 是不是素数
31. //  $n - 1 = x * 2^t$  中间使用二次判断
32. // 是合数返回 true, 不一定是合数返回 false
33. bool check(long long a, long long n, long long x, long long t){
34.     long long ret = pow_mod(a, x, n);
35.     long long last = ret;
36.     for(int i = 1; i <= t; i++){
37.         ret = mult_mod(ret, ret, n);
38.         if(ret == 1 && last != 1 && last != n-1) return true; // 合数
39.         last = ret;
40.     }
41.     if(ret != 1) return true;
42.     else return false;
43. }
44. //*****
45. // Miller_Rabin 算法
46. // 是素数返回 true, (可能是伪素数)
47. // 不是素数返回 false
48. //*****
49. bool Miller_Rabin(long long n){
50.     if( n < 2) return false;
51.     if( n == 2) return true;
52.     if( (n&1) == 0) return false; // 偶数
53.     long long x = n - 1;
54.     long long t = 0;
55.     while( (x&1) == 0 ){ x >>= 1; t++; }
56.     srand(time(NULL)); /* ***** */
57.     for(int i = 0; i < S; i++){
58.         long long a = rand()%(n-1) + 1;
59.         if( check(a, n, x, t) )
60.             return false;
61.     }
62.     return true;
63. }
64. //*****
65. // pollard_rho 算法进行质因数分解
66. //*****
67. long long factor[100]; // 质因数分解结果 (刚返回时是无序的)
68. int tol; // 质因数的个数, 编号 0~tol-1
69. void findfac(long long n, int k){
70.     if(n == 1) return;
71.     if(Miller_Rabin(n))
72.     {

```

```

73.         factor[tol++] = n;
74.         return;
75.     }
76.     long long p = n;
77.     int c = k;
78.     while( p >= n)p = pollard_rho(p,c--); //值变化, 防止死循环 k
79.     findfac(p,k);
80.     findfac(n/p,k);
81. }

```

板子来自于 kuangbin, 经典例题有 POJ 1811, 给出一个 $N(2 \leq N < 2^{54})$, 如果是素数, 输出 "Prime", 否则输出最小的素因子

5.4 欧拉函数

5.4.1 欧拉函数定义

在数论, 对正整数 n , 欧拉函数是小于 n 的正整数中与 n 互质的数的数目。

5.4.2 欧拉函数实现

```

1. int calc(int n)
2. { //返回小于 n 与 n 互质数个数
3.     int ans=n;
4.     for(int i=2;i*i<=n;i++)
5.     {
6.         if(n%i==0)
7.         {
8.             ans=ans/i*(i-1);
9.             while((n/=i)%i==0){}
10.        }
11.    }
12.    if(n>1)
13.        ans=ans/n*(n-1);
14.    return ans;
15. }

```

5.4.3 欧拉函数打表

若 p 是素数, 则 $\varphi(p) = p - 1$

证明: 由于 p 是素数, 因此 Z_p 是一个域, 从而 $Z_p^* = \overline{1}, \overline{2}, \overline{3}, \dots, \overline{p-1}$, 于是 $\varphi(p) = p - 1$

设 p 是素数, 则对于任一正整数 r , 有

$$\varphi(p^r) = p^{r-1}(p - 1)$$

```

1. int euler[100001]; //处理多个欧拉函数
2. void geteuler()
3. {

```

```

4.     memset(euler,0,sizeof(euler));
5.     euler[1]=1;
6.     for(int i=2;i<=100000;i++)
7.         if(!euler[i])
8.             for(int j=i;j<=100000;j+=i)
9.                 {
10.                    if(!euler[j])
11.                        euler[j]=j;
12.                        euler[j]=euler[j]/i*(i-1);
13.                }
14. }

```

5.5 组合数

这里写处理大数用Lucas定理处理。

代码实现可简单理解为 $C_n^m = C_{n/p}^{m/p} * C_{n \bmod p}^{m \bmod p} \bmod p$

```

1. LL fac[N];
2. void getFac()
3. { //构造阶乘
4.     fac[0]=1;
5.     for(int i=1;i<1000000;i++)
6.     {
7.         fac[i]=fac[i-1]*i%MOD;
8.     }
9. }
10. LL quickPowMod(LL a,LL b,LL mod)
11. { //快速幂
12.     LL res=1;
13.     while(b)
14.     {
15.         if(b&1)
16.             res=res*a%mod;
17.         b>>=1;
18.         a=a*a%mod;
19.     }
20.     return res;
21. }
22. LL getC(LL n,LL m,LL mod)
23. { //获取 C(n,m)%mod
24.     if(m>n) return 0;
25.     return fac[n]*(quickPowMod(fac[m]*fac[n-m]%mod,mod-2,mod))%mod;
26. }
27. LL Lucas(LL n,LL m,LL mod)

```

```

28. { // 卢卡斯定理
29.     if(m==0)
30.         return 1;
31.     return getc(n%mod,m%mod,mod)*Lucas(n/mod,m/mod,mod)%mod;
32. }
33. int main()
34. {
35.     getFac();
36.     LL n,m;
37.     scanf("%lld%lld",&n,&m);
38.     printf("%lld\n",Lucas(n,k,MOD));
39.     return 0;
40. }

```

5.6 快速幂

以 $O(\log n)$ 的时间复杂度求 $a^n \bmod c$ 。

```

1. LL quickPowMod(LL a,LL b,LL mod)
2. { // 快速幂
3.     LL res=1;
4.     while(b)
5.     {
6.         if(b&1)
7.             res=res*a%mod;
8.         b>>=1;
9.         a=a*a%mod;
10.    }
11.    return res;
12. }

```

5.7 扩展欧几里得

5.7.1 扩展欧几里得简介

扩展欧几里得算法是欧几里得算法（又叫辗转相除法）的扩展。除了计算 a 、 b 两个整数的最大公约数，此算法还能找到整数 x 、 y （其中一个很可能是负数）。

通常谈到最大公因子时，我们都会提到一个非常基本的事实：给予二整数 a 与 b ，必存在有整数 x 与 y 使得 $ax + by = \gcd(a, b)$ 。有两个数 a, b ，对它们进行辗转相除法，可得它们的最大公约数——这是众所周知的。然后，收集辗转相除法中产生的式子，倒回去，可以得到 $ax + by = \gcd(a, b)$ 的整数解。

首先要知道两个定理。定理一： $\gcd(a, b) == \gcd(b, a \% b)$ ；和定理二： $a * x + b * y = \gcd(a, b)$ 一定存在解，这个又叫又叫裴蜀定理，或贝祖定理。

5.7.2 扩展欧几里得实现

```

1. #include<bits/stdc++.h>
2. using namespace std;
3. int exgcd(int a,int b,int &x,int &y)
4. {
5.     if(b==0)
6.     {
7.         x=1;
8.         y=0;
9.         return a;
10.    }
11.    int r=exgcd(b,a%b,x,y);
12.    int temp=x;
13.    x=y;
14.    y=temp-(a/b)*y;
15.    return r;
16. }
17. int main()
18. {
19.     int a,b,c,x,y,gcd,t;
20.     scanf("%d%d%d",&a,&b,&c);
21.     gcd=exgcd(a,b,x,y);
22.     t=c/gcd;
23.     int x1=x*t,y1=y*t;
24.     printf("%d %d\n",x1,y1);
25.     return 0;
26. }

```

5.8 除法取模与逆元

5.8.1 逆元

众所周知，取模运算里面没有办法用除法，所以如何进行除法运算就成了一个问题了，这个时候逆元就出现了，一个数在模意义下乘以某个数再%这个模意义下的数得到的答案是1，那么‘某个数’就是这个数的逆元。

逆元可以代替除法，除以这个数就等于乘以这个数的逆元。

5.8.2 费马小定理

如果 p 是一个质数，而整数 a 不是 p 的倍数，则有 $a^{p-1} \equiv 1 \pmod{p}$ 。

也可以写成 $a^p \equiv a \pmod{p}$ 。

用数学归纳法可以证明。

5.8.3 运用

我们如果想要除以 a 的话，只需要乘以它的逆元，也就是 a^{p-2} ，这个用快速幂可以解决。

5.8.4 费马小定理局限性

因为费马小定理只适用于模数是质数的情况，所以也只能解决模数是质数情况下的逆元，不过在大部分的题目中还是很实用的。

5.9 威尔逊定理

5.9.1 威尔逊定理

$(p-1)! \equiv -1 \pmod{p}$ 是 p 为质数的充要条件

5.9.2 威尔逊定理证明

充分性:

若 p 不是质数, $\gcd((p-1)!, p) > 1$, 这样的话一定得不到 $\text{mod } p = p-1$

必要性:

设 $x^2 \equiv 1 \pmod{p}$ 解得 $x = 1$ 或 $p-1$, 所以在 $[1, p-1]$ 中只有 1 和 $p-1$ 的逆元为自己, 而其他的都存在与自己不相等的逆元, 使得 $(p-1)! = 1 * (p-1) \equiv -1 \pmod{p}$

6 STL 库

6.1 unique 函数

unique 用于去重, 将一个函数中去重的函数移到前面来, 后面的不做更改, 具体举例

```
1. int main()
2. {
3.     vector<int> v={1,2,2,3,3,4,4};
4.     for (auto i:v) cout<<i<<" ";
5.     cout<<endl;
6.     unique(v.begin(),v.end());
7.     for (auto i:v) cout<<i<<" ";
8.     cout<<endl;
9.     return 0;
10. }
11. //Output
12. 1 2 2 3 3 4 4
13. 1 2 3 4 3 4 4
```

由此可以看出, 一共有四位不同的元素, 所以前四位是保证不重样的, 然后后面三位原封不动的放在原地。

以上是 vector 的用法, 接下来看看数组的用法

```
1. int main()
2. {
3.     cin>>n;
4.     for (int i=0;i<n;i++) cin>>a[i];
5.     int len=unique(a,a+n)-a;
6.     cout<<len<<endl;
7.     for (int i=0;i<n;i++) cout<<a[i]<<" ";
8.     cout<<endl;
```

```

9.     for (int i=0;i<len;i++) cout<<a[i]<<" ";
10.    return 0;
11. }
12. //Input
13. //6
14. //2 2 3 3 5 5
15. //Output
16. //3
17. //2 3 5 3 5 5
18. //2 3 5

```

由此可见，确实是能保证将去重的放置前面。
但是，如果 *unique* 从 $a + 1$ 到 $a + 1 + n$ ，就必须在后面 -1 才能得到正确的 *len*

6.2 map 的遍历

map 的用法大家都会，就这里写一下 map 是如何遍历的，因为我们定义 map 的时候是这样的

```
1. map<int,int> mp
```

map 相当于一个 `pair<int,int>` 存入 map 中，所以我们用 auto 的时候得到的是一个 pair 类型的。

```

1. for (auto i:mp)
2.     cout<<i.first<<" "<<i.second<<endl;

```

如果想要改变，则需要要在 *i* 前面加上 '&'

6.3 lower_bound&upper_bound

在从小到大的排序数组中：

lower_bound(begin,end,num): 从数组的 begin 位置到 end-1 位置二分查找第一个大于或等于 num 的数字，找到返回该数字的地址，不存在则返回 end。通过返回的地址减去起始地址 begin,得到找到数字在数组中的下标。

upper_bound(begin,end,num): 从数组的 begin 位置到 end-1 位置二分查找第一个大于 num 的数字，找到返回该数字的地址，不存在则返回 end。通过返回的地址减去起始地址 begin,得到找到数字在数组中的下标。

在从大到小的排序数组中，重载 lower_bound()和 upper_bound()

lower_bound(begin,end,num,greater<type>()): 从数组的 begin 位置到 end-1 位置二分查找第一个小于或等于 num 的数字，找到返回该数字的地址，不存在则返回 end。通过返回的地址减去起始地址 begin,得到找到数字在数组中的下标。

upper_bound(begin,end,num,greater<type>()): 从数组的 begin 位置到 end-1 位置二分查找第一个小于 num 的数字，找到返回该数字的地址，不存在则返回 end。通过返回的地址减去

起始地址 `begin`,得到找到数字在数组中的下标。

6.4 其他 STL

上面三点是我总结一般会错的, STL 很强大, 例如我们经常用的 `priority_queue`, `deque`, `list`, `vector` 等, 还有迭代器的用法需要注意。有些相对简单的就不必写了, 希望以后自己多多总结, 多多反思, 用好 STL。